

07Lab

Stochastic modelling: uncertainty, probability and frequencie

Mirko Viroli

`mirko.viroli@unibo.it`

C.D.L. Magistrale in Ingegneria e Scienze Informatiche
ALMA MATER STUDIORUM—Università di Bologna, Cesena

a.a. 2025/2026

One slide sum-up on Stochastic Modelling

Concepts

- evolve our reference meta-meta-model of transition systems in two directions
- DTMC: adding probabilities to “quantify non-determinism”, still in a discrete-time setting
 - ▶ this will later be extended to deal with rewards, hence Reinforcement Learning
- CTMC: adding markovian rates to “quantify continuous and probabilistic passage of time”
 - ▶ this will later be extended to deal with event-based simulations

CTMC analysis

- a CTMC can be analysed by probabilistically drawing possible traces: that’s actually a simulation
- will later introduce the concept of probabilistic approximate model-checking

CTMC-based meta-models

- adding rates to PN gives SPN (rates can linearly depend on number of tokens in incoming places)
- SPN are interesting: they model chemistry (up to bio-chemistry)
- adding distribution in DAP: possibly also distributed networks

Starting point and goals

References

- repo: from virtuale, folder u07 (<https://github.com/mviroli/asmd-public-models>)

General goals

- get acquainted with CTMCs, their simulation, and meta-models
- play with SPN, play with DAP
- reason about probability, time, and testing

Operational Steps

Step 1 – Download and check

- Download the lab repository
- Import it in IntelliJ as an SBT project
- Get acquainted with all parts
 - ▶ `utils`: just the `MSet` abstraction, and also others
 - ▶ `modelling`: the actual API/DSL definition with `CTMC`, `CTMCSimulation` (meta-meta-model); then `SPN` and `DAP` (meta-model)
 - ▶ `examples`: extensional example `StochasticChannel`; intensional by Petri Nets `StochasticMutualExclusion` and by `DAP` `DAPGossip`
 - ▶ `test`: checks and tests
- Check that all Scala tests and checks pass and why

Step 2 – “Implement” stochastic readers & writers as of slides, and play with it

- Perform some run: is it enough to give evidence of correctness of expected behaviour?

Step 3 – Play with `DAPGossip`

- Implement and simulate a system where gossip-like broadcasts are used to send a message and get a reply from a node (the reply is eventually received only if a target is ever reached)

Tasks

SIMULATOR

Take the communication channel CTMC example in `StochasticChannelSimulation`. Compute the average time at which communication is done—across n runs. Compute the relative amount of time (0% to 100%) that the system is in fail state until communication is done—across n runs. Extract an API for nicely performing similar checks.

GURU

Check the SPN module, that incorporates the ability of CTMC modelling on top of Petri Nets, leading to Stochastic Petri Nets, and Stochastic Readers & Writers. Try to study how key parameters/rate influence average time the system is in read or write state.

CHEMIST

SPNs can be used to simulate dynamics of chemical reactions. Experiment with it. E.g.: search the “Brussellator” chemical reaction on wikipedia, or other similar ones: it oscillates! Try to come up with a chemical reaction which oscillates.

RANDOM-UNIT-TESTER

How do we unit-test with randomness? And how we test at all with randomness? Think about this in general. Try to create a repeatable unit test for `Statistics` as in `utils.StochasticSpec`.

PROBABILITY-LLM

We know that LLMs/ChatGPT can arguably help in write/improve/complete/implement/reverse-engineer standard ProgLangs. But is it of help in taking into account probability? Seemingly, it is shorty fails. But with the proper prompt, it might say something reasonable.